

SO1 - Sobre el Algoritmo de Peterson

29 de marzo de 2022

1. El Algoritmo

El algoritmo de Peterson, publicado en 1981, es una solución en software al problema de exclusión mútua. Si bien la primer solución se debe a Dekker en 1959 (según Dijkstra [n.d., 1965]), el algoritmo de Peterson es notablemente más simple, y su análisis es por lo tanto más fácil.

Vamos a demostrar que el algoritmo de Peterson garantiza:

1. exclusión mútua: no pueden ambos procesos entrar a su región crítica (RC) a la vez
2. starvation freedom: ningún procesador queda bloqueado indefinidamente

Trabajaremos con la siguiente versión del algoritmo. Asumimos dos procesadores independientes, cada uno corriendo su respectivo fragmento.

```
f1 = true
turn = 2
while (f2 && turn == 2)
    /* esperar */;
CRIT;
f1 = false
```

Código de P1

```
f2 = true
turn = 1
while (f1 && turn == 1)
    /* esperar */;
CRIT;
f2 = false
```

Código de P2

Nota: obviamente, se asume que la RC (denotada por CRIT;) no modifica ninguna de las variables relevantes del algoritmo, i.e. `f1`, `f2`, y `turn`.

1.1. Demostración de Exclusión Mútua

Primero, notemos que basta con analizar una sola corrida de ambos procesadores. Por más que ambos procesadores estén continuamente intentando acceder a su RC en un bucle (ej. jardines ornamentales), el análisis para cada iteración es el mismo que para la primera dado que los procesadores restauran el estado de las flags al terminar (y el valor inicial de `turn` es siempre irrelevante).

Ahora, asumiremos que ambos procesadores pudieron entrar a su RC a la vez y llegaremos a una contradicción. Vamos a pensar en el momento exacto en que un procesador entra a su RC cuando ya el otro estaba en su RC. Sin pérdida de generalidad, asumamos que P1 es el procesador que entra último.

La condición del **while** de espera de P1 acaba de fallar. Si esto ocurrió porque `f2 == false`, es claro que P2 no puede estar en su RC (pues escribe `f2` en `true` antes de entrar, y la vuelve a `false` sólo luego de terminarla).

Por otro lado, puede ser que la condición haya fallado porque `turn == 1`. Dado que P1 setea `turn == 2` antes del **while**, necesariamente tiene que haber ocurrido que P2 ejecutó su asignación a `turn` entre esta sentencia y el **while**. **Notar que esto implica que `f1` es `true` al momento que P2 asigna a `turn`**, y por lo tanto también cuándo P2 llega a su **while**. Entonces, P2 no pudo haber avanzado de su `while` de espera al no estar en su turno, y también llegamos a una contradicción. $\square$

Nota: Al ser un algoritmo tan simple, también es factible analizar por fuerza bruta todas las trazas posibles.

1.2. Demostración de Starvation-Freedom

(No vamos a ser tan formales en esta sección.)

Lo que debemos demostrar es que cada procesador eventualmente hace progreso. Para ello, naturalmente asumimos que la RC eventualmente termina, y que cada procesador eventualmente ejecuta alguna instrucción (aunque sea de manera muy lenta).

Si ambos procesadores llegan al **while** de espera, ¿puede ocurrir que ambos queden trabados allí? No, porque la variable `turn` le da el paso a exactamente uno de ellos. Esto es suficiente para demostrar la ausencia de *deadlock*: siempre *al menos uno* de los procesadores puede progresar.

Yendo más allá, el algoritmo de Peterson también cumple *starvation freedom*, i.e. la propiedad de que *todo* procesador eventualmente avanza. Para convencerse de esto, asumamos que P1 entró a su RC y P2 quiere entrar, quedando bloqueado en el **while**. Eventualmente, P1 terminará su RC y bajará su flag. En ese momento, P2 puede avanzar ya que es necesariamente su turno. Además, P1 *no puede avanzar* hasta que P2 termine, debido a que no es su turno. Por ello, es imposible que un sólo procesador acapare el uso del mutex indefinidamente.

2. Si seteamos `turn` primero

Consideremos cambiar el código para setear `turn` antes que los flag `f0` y `f1`:

```
turn = 2
f1 = true
while (f2 && turn == 2)
    /* esperar */;
CRIT;
f1 = false
```

Código de P1

```
turn = 1
f2 = true
while (f1 && turn == 1)
    /* esperar */;
CRIT;
f2 = false
```

Código de P2

En este caso, sí existe una forma de romper exclusión mutua, dada por la siguiente traza:

<pre> (P1) turn = 2 f1 = true f2 && t == 2 evalúa a false CRIT; </pre>	<pre> (P2) turn = 1 f2 = true f1 && t == 1 evalúa a false CRIT; </pre>
---	---

La intuición es que P2 pudo avanzar (pisando el “desempate” de P1) antes de que P1 declare su intención de entrar vía `f1`. Luego, P1 avanza dado que “es su turno”.

La prueba anterior no aplica a esta versión del algoritmo: la parte que falla es el segundo caso, cuando P1 entra a la RC porque `turn == 1`. Con esta variante, sabemos que P2 ejecutó `turn = 1` en algún punto posterior al `turn = 2` de P1, pero esto pudo haber ocurrido antes de que se escriba `f1 = true`. En ese caso, pudo haber entrado a su RC.

3. Consistencia secuencial y *fences*

En computadoras modernas, incluso la versión correcta del algoritmo puede fallar y dejar a ambos procesadores entrar a su RC. ¿Cómo puede ser, si demostramos que era imposible? La respuesta es que nuestra prueba dependía de una asunción tácita: la *consistencia secuencial*, es decir que el comportamiento del sistema multiprocesador es una combinación (*intercalamiento* o *interleaving*) de las instrucciones individuales de cada procesador. Más concretamente, asumimos 1) que las operaciones de cada procesador se realizan en el orden especificado, y 2) que los stores son inmediatamente visibles al otro procesador.

Si bien ambas son asunciones razonables, resultan no ser ciertas en procesadores modernos (ej. x86). Estos implementan *ejecución fuera de orden* y *store buffering*.

Con ejecución fuera de orden, puede ser que algunas instrucciones se hagan antes de lo estipulado en el código si la CPU se encuentra bloqueada para seguir el pipeline de instrucciones secuencialmente. Ejemplo:

```

divl %ecx          ; eax := eax/ecx
movl arr(%eax), dest
movl flag, %ebx

```

Supongamos que la división puede tardar alrededor de 20 ciclos. El primer `movl` depende del resultado de la división, con lo cual no podemos ejecutarlo sin esperar al resultado de la división. Sin embargo, el segundo `movl` no depende de ninguna instrucción previa, y la CPU puede detectar esto y ejecutarlo *mientras* se espera el resultado de la división. Estos reordenamientos pueden ocurrir con cientos de instrucciones de por medio.

No todos los reordenamientos son permitidos, las reglas *dependen de cada arquitectura*. En x86/x64, se garantiza que los únicos reordenamientos que pueden ocurrir son mover *loads* (lecturas de memoria) antes de *stores* (escrituras a memoria), obviamente mientras no sean a la misma dirección. Todo otro reordenamiento está prohibido (en particular, los stores no se reordenan entre sí, con lo cual la CPU no puede convertir nuestro algoritmo al de la Sección 2).

Considerar en el algoritmo de Peterson qué pasa si la lectura de los flag en la condición del **while** se ejecuta como primer paso (y verificar que está permitido por las reglas de x86/x64).

Por otro lado, los sistemas multiprocesador no garantizan que sus cachés son consistentes. Cuando un

procesador escribe en una dirección de memoria, esto no es inmediatamente visible a los otros procesadores. Nuevamente, esto implica que ambos procesadores ejecutando el algoritmo de Peterson pueden avanzar en sincronía y entrar a sus RC. (En x86/x64, esto se debe en particular al *store-buffering*.)

Si bien la CPU reordena y no mantiene cachés coherentes, hace la burocracia necesaria para *fingir* que las operaciones se realizan secuencialmente. Dentro de cada procesador, no podemos observar estos efectos. Al escribir primitivas de sincronización, esta ilusión se rompe, y debemos hacer algunos cambios explícitos.

En x86, una forma de paliar estos efectos es usar un *fence* o *barrera de memoria*. Un fence (instrucción `mfence`) causa que la CPU:

1. no reordene instrucciones a través del mismo
2. garantice que todas las escrituras previas al fence son visibles a todos los procesadores antes de continuar

De esta forma, podemos arreglar el algoritmo así:

```
f1 = true
turn = 2
asm("mfence");
while (f2 && turn == 2)
    /* esperar */;
CRIT;
f1 = false
```

Código de P1

```
f2 = true
turn = 1
asm("mfence");
while (f1 && turn == 1)
    /* esperar */;
CRIT;
f2 = false
```

Código de P2

Nota: esta sección es sólo relevante para sistemas multiprocesador. Si estuviéramos corriendo en un sistema con un único procesador, no observaremos estos problemas. Notar que aun con un sólo procesador necesitamos usar algún sistema de exclusión mutua ya que los procesos pueden interrumpirse en cualquier momento. (En Linux, puede usarse `taskset` para forzar un proceso a usar un único núcleo/procesador y verificarlo, en ese caso el algoritmo funciona sin fences, por más que el CPU implementa reordering y store-buffering.)

3.1. Optimizaciones del Compilador

Otro problema independiente son las optimizaciones del compilador. En general, el compilador asume que las variables no cambian espontáneamente. Es decir, si tenemos el fragmento:

```
v = 1;
/* ... mucho código que no modifica a `v` */
x = v;
```

el compilador está autorizado a optimizar la última asignación a `x=1`. Nuevamente, esto nos puede traer problemas (explicar cómo, y verificarlo usando `-O3`).

Estas optimizaciones ocurren *en tiempo de compilación*, antes de correr el programa. Puede inspeccionarse el código assembly generado para determinar si fueron hechas o no. Por otro lado, los problemas de la sección anterior ocurren en *tiempo de ejecución*.

Para evitar esto, se usa en general la palabra clave **volatile**. Usada en declaraciones de variables le dice al compilador que la variable puede cambiar espontáneamente, sin acción del programa. Es decir, si `v` fuera declarada como **volatile**, la optimización anterior ya no está permitida.

4. Instrucciones de Sincronización (x86)

Usualmente, no se usa el algoritmo de Peterson para sincronizar procesos. Tampoco el de Lamport, ni ningún otro que funcione solamente por memoria compartida. En cambio, se usan instrucciones especiales provistas por la arquitectura que permiten sincronizar de manera más eficiente.

El ejemplo clásico es usando una instrucción *compare-and-swap* (CAS) atómica, la cual es provista por varias arquitecturas. Genéricamente, la idea de `CAS(l,a,b)` es que lee la dirección de memoria `l`, y si contiene un valor igual a `a`, lo sobrescribe con `b`, pero no lo modifica en otro caso. A la vez, “devuelve” un booleano que indica si la escritura fue realizada o no. De esta forma, podemos implementar mutexes usando una variable en memoria que valga 0 cuando el mutex está libre y 1 cuando está tomado. Con esto, `lock()` puede implementarse como `while (!CAS(l, 0, 1));`.

4.1. CAS en x86: `cmpxchgl`

La arquitectura x86 provee `cmpxchgl`¹. La instrucción toma dos operandos: una “fuente” y un “destino” (usualmente una dirección de memoria). Además, lee el valor del registro `%eax`. El funcionamiento es el siguiente: si el destino contiene un valor igual a `%eax` entonces se escribe el valor de la fuente en el destino. Si el destino contiene otro valor, entonces este valor se lee al registro `%eax`. En pseudocódigo, es algo así²:

```
cmpxchgl src, dest =
    if (*dest == eax)
        *dest = src;
    else
        eax = *dest;
```

La instrucción `cmpxchgl` *no es atómica* por sí misma, pero puede usarse el prefijo `lock` de x86 para que sí lo sea. No nos sirve de mucho usarla sin el prefijo para hacer sincronización, así que lo usaremos siempre.

Para más información ver Intel [2016].

Con esta instrucción, podemos implementar un mutex de la siguiente forma:

```
volatile uint32_t lk;
void lock() {
    do {
        asm("movl $0, %%eax\n\t"
            "movl $1, %%ebx\n\t"
            "lock; cmpxchgl %%ebx, %0\n\t"
            "movl %%eax, %1"
            : "+m"(lk), "=m"(no_lock)
            : "eax", "ebx", "memory");
    } while (no_lock);
}
void unlock() { lk = 0; }
```

La función `lock` usa `cmpxchgl` para setear la variable `lk` en 1 si estaba en 0, y no hacer nada en caso contrario. Luego mueve `%eax` a la variable `no_lock`. En el caso de haber seteado `lk` (i.e. haber tomado el lock), `%eax` permanece en 0, y el `do..while` termina. Si no fue seteado, `lk` está en un valor distinto a 0, el cual es leído a `%eax` y luego a `no_lock`, con lo cual se sigue intentando tomar el lock.

¹Esta es la variante para un “long” (32 bits). Existen otras versiones de `cmpxcgh`.

²La instrucción también setea el flag `zf` del procesador según el caso, pero no usaremos esto aquí.

El código assembler inline parece más complejo de lo que es. Solamente está seteando `%eax` en 0, `%ebx` en 1, y ejecutando `lock; cmpxchgl %ebx, 1k` (verificar viendo el assembler generado por el módulo). Finalmente, mueve el valor de `%eax` a la variable `no_lock`. El resto del texto le da suficiente información al compilador sobre cuales registros se leen/modifican con esta sentencia, para permitirle alocar registros y posiblemente optimizar.

Esta implementación ya está mucho más de cerca de una implementación real de mutexes (e.g. la de la librería `pthread`). El principal ingrediente faltante es “dormir” al proceso si no pudo adquirir el lock, en lugar de hacer busy waiting. Esto ya requiere soporte (y depende de) el sistema operativo.

5. Ejercicios

Ejercicio 1 ¿Puede fallar la siguiente aserción? Explique. Si puede fallar, arregle el programa.

```
#include <stdio.h>
#include <pthread.h>
#include <assert.h>

int x=0, y=0, a=0, b=0;

void * foo(void *arg) { x = 1; a = y; return NULL; }
void * bar(void *arg) { y = 1; b = x; return NULL; }

int main()
{
    pthread_t t0, t1;

    pthread_create(&t0, NULL, foo, NULL);
    pthread_create(&t1, NULL, bar, NULL);

    pthread_join(t0, NULL);
    pthread_join(t1, NULL);

    assert (a || b);

    return 0;
}
```

Ejercicio 2 ¿Puede fallar la siguiente aserción? Explique. Si puede fallar, arregle el programa.

```
#include <stdio.h>
#include <pthread.h>
#include <assert.h>

volatile int x = 0;
volatile int y = 0;

void * wr(void *arg)
{
    x = 123;
    y = 1;
    return NULL;
}

void * rd(void *arg)
```

```

{
    while (!y)
        ;
    assert(x == 123);
    return NULL;
}

int main()
{
    pthread_t t0, t1;

    pthread_create(&t0, NULL, wr, NULL);
    pthread_create(&t1, NULL, rd, NULL);

    pthread_join(t0, NULL);
    pthread_join(t1, NULL);

    return 0;
}

```

Ejercicio 3 Explique el funcionamiento del siguiente programa, cuando es compilado sin optimizaciones y con optimizaciones (-O3 para gcc).

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int go = 0;

void * th(void *arg)
{
    while (!go) ;
    printf("Listo!\n");
    return NULL;
}

int main()
{
    pthread_t t;
    pthread_create(&t, NULL, th, NULL);
    sleep(1);
    go = 1;
    pthread_join(t, NULL);
    return 0;
}

```

Ejercicio 4 Para la versión ingenua (sin sincronización) del jardín ornamental, ¿qué pasa cuándo compilamos con optimizaciones? Pista: ver el assembler generado.

Ejercicio 5 En la siguiente implementación del jardín ornamental, agregue estratégicamente algunos sleep para obtener el mínimo valor posible de visitantes. Puede usar condicionales.

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

```

```

const int N = 1000000;
int visitantes;

void * proc(void *arg)
{
    int i;
    int id = arg - (void*)0;
    for (i = 0; i < N; i++) {
        int c;
        /* sleep? */
        c = visitantes;
        /* sleep? */
        visitantes = c + 1;
        /* sleep? */
    }
    return NULL;
}

int main()
{
    pthread_t t0, t1;

    pthread_create(&t0, NULL, proc, 1 + (void*)0);
    pthread_create(&t1, NULL, proc, 2 + (void*)0);

    pthread_join(t0, NULL);
    pthread_join(t1, NULL);

    printf("Cantidad = %i\n", visitantes);

    return 0;
}

```

Ejercicio 6 Determine si la siguiente variante del jardín ornamental es correcta (para x86). Explique.

```

#include <stdio.h>
#include <pthread.h>
#include <stdint.h>
#include <assert.h>

const int N = 10000000;
volatile int visitantes;

void * proc(void *arg)
{
    int i;
    for (i = 0; i < N; i++)
        asm("lock; incl %0" : "+m"(visitantes) : : "memory");

    return NULL;
}

int main()
{
    pthread_t t0, t1;

    pthread_create(&t0, NULL, proc, 1 + (void*)0);
    pthread_create(&t1, NULL, proc, 2 + (void*)0);
}

```

```

    pthread_join(t0, NULL);
    pthread_join(t1, NULL);

    printf("Cantidad = %i\n", visitantes);
    assert(visitantes == 2 * N);

    return 0;
}

```

Ejercicio 7 En Linux, puede usarse `taskset` para forzar a que ambos hilos del jardín ornamental corran en el mismo procesador. Si hacemos esto,

1. ¿es aún necesaria la sincronización? (i.e. el algoritmo de Peterson u otro)
2. ¿son necesarias las barreras de memoria?
3. Si usamos inline assembly para incrementar `visitantes` con `incl` (sin prefijo `lock`), ¿es necesaria la sincronización y/o barreras de memoria?

Ejercicio 8 Compare la performance del jardín ornamental (para una misma cantidad de visitantes totales) para 1) la solución incorrecta sin sincronización 2) la solución con el algoritmo de Peterson 3) usando un solo molinete sin multithreading.

Ejercicio 9 Para el algoritmo de Peterson, ¿puede ubicarse el `mfence` entre la asignación al flag y la asignación a `turn`? Justifique.

6. Otras lecturas

El paper original de Peterson, que incluye el algoritmo y su prueba de correctitud en 2 páginas, puede encontrarse en Peterson [1981].

El capítulo 7 de Herlihy and Shavit [2008] tiene algunos detalles sobre la implementación real de algoritmos de sincronización.

Para mucho más detalle sobre barreras de memoria y detalles micro-arquitectónicos ver McKenney [2010].

Alagarsamy [2003] discute diferencias entre las distintas soluciones a estos problemas.

Para más información de la propiedad de starvation freedom, ver Raynal [2012].

Referencias

- K. Alagarsamy. Some myths about famous mutual exclusion algorithms. *SIGACT News*, 34(3):94–103, sep 2003. ISSN 0163-5700. doi: 10.1145/945526.945527. URL <https://doi.org/10.1145/945526.945527>.
- Edsger W. Dijkstra. Cooperating sequential processes, technical report (EWD-123). Technical report, 1965. URL <http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF>.
- Edsger W. Dijkstra. Over de sequentialiteit van procesbeschrijvingen (EWD-35). Circulated privately, n.d. URL <http://www.cs.utexas.edu/users/EWD/ewd00xx/EWD35.PDF>.

Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2008. ISBN 0123705916.

Intel. *Intel 64 and IA-32 Architectures Software Developer's Manual: Instruction Set Reference, A-Z, Volume 2 (2A, 2B, 2C & 2D)*, 2016. URL <https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-instruction-set-reference-manual-325383.pdf>.

Paul McKenney. Memory barriers: a hardware view for software hackers. aug 2010. URL <http://www.rdrop.com/~paulmck/scalability/paper/whymb.2010.06.07c.pdf>.

Gary L. Peterson. Myths about the mutual exclusion problem. *Inf. Process. Lett.*, 12(3):115–116, 1981. URL [http://dx.doi.org/10.1016/0020-0190\(81\)90106-X](http://dx.doi.org/10.1016/0020-0190(81)90106-X).

M. Raynal. *Concurrent Programming: Algorithms, Principles, and Foundations*. Springer Berlin Heidelberg, 2012. ISBN 9783642320279. URL <https://books.google.com.ar/books?id=0a1AAAAAQBAAJ>.